MINOR PROJECT REPORT

OF

WEB DEVELOPMENT / FULL STACK

ON

FINDIT

SMART CAMPUS LOST & FOUND PLATFORM

SUBMITTED IN PARTIAL FULFILLMENT OF THE DEGREE

OF

BACHELOR OF TECHNOLOGY

IN

Computer Science and Engineering

Submitted By:

Taran Singh
Roll No.: 12401067

Prince
Roll No.: 12401062

Under the guidance of Dr. Harpreet Kaur

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PUNJABI UNIVERSITY, PATIALA - 147002

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us throughout the completion of this minor project. Their guidance, technical insight, and continuous encouragement were instrumental in helping us design, implement, and document a complete full-stack web application from first requirements through a working campus demonstration.

We are deeply thankful to our project mentor, Dr. Harpreet Kaur, and to the faculty of the Department of Computer Science & Engineering, Punjabi University, Patiala, for invaluable feedback that helped us refine the product scope, the layered API design, the explainable matching heuristic, and the claim-moderation workflow. Their expertise was crucial in helping us overcome challenges related to authentication, data modelling, and local deployment.

Finally, we thank our peers and families for unwavering support and motivation during development. This project has been a significant learning milestone in modern web development, and we are grateful for the opportunity to practise the full path from problem statement to a usable Next.js and Express application backed by MongoDB.

Thank you all for your support.

Taran Singh
Roll No.: 12401067

Prince
Roll No.: 12401062

ABSTRACT

Lost belongings on a university campus are usually announced in scattered group chats and notice boards. Those posts are hard to search, easy to miss, and awkward when a student must share a private identifying detail in public. FindIt addresses this gap with a smart campus lost-and-found platform: students report lost or found items, search approved listings, review short explanations when another report looks similar, and follow a moderated claim path that keeps ownership proof off the public record.

The system is implemented as two cooperating applications in one repository. The browser interface is built with Next.js App Router, React, TypeScript, and Tailwind CSS. Business operations are handled by an Express API organised into controllers, services, and repositories, with Mongoose models persisted in MongoDB. Sessions use an HTTP-only cookie; the API stores only a SHA-256 hash of the session identifier. Registration always creates a student role. Administrators are created by a seed script and can moderate reports, review claims, and inspect activity history.

Match suggestions are a capped, explainable heuristic rather than a machine-learning probability. Default weights favour category (0.35) and meaningful word overlap in title and description (0.35), with smaller contributions from location words (0.15) and dates inside a fourteen-day window (0.15). Candidates below a minimum score of 0.35 are dropped. Claims move through pending, approved, rejected, and cancelled states; approval can mark an item resolved. Automated tests use Vitest and Supertest against an in-memory MongoDB.

As a minor project in Web Development / Full Stack, this work demonstrates a complete campus-oriented web workflow: authentication, moderated content, searchable listings, explainable matching, claim review, and administrator oversight — packaged for local demonstration with seed data and documented setup steps.

CANDIDATE'S DECLARATION

We, "TARAN SINGH" (Roll No. 12401067) and "PRINCE" (Roll No. 12401062), hereby declare that we have undertaken this Minor Project on "FindIt — Smart Campus Lost & Found" in partial fulfillment of the requirements for the award of the degree of B.Tech (Department of Computer Science & Engineering). The work presented in this project report is an authentic record of our own work carried out under the guidance of Dr. Harpreet Kaur. We sincerely thank our mentor for valuable guidance, support, and encouragement throughout the project.

Signature of the Students

Taran Singh _____ Prince _____

The minor project Viva-Voce Examination has been held on _____ and accepted.

Signature of Mentor

Dr. Harpreet Kaur _____

LIST OF FIGURES

Figure 4.1: Report status path used during moderation 7
Figure 5.1: End-to-end framework of FindIt 9
Figure 5.2: Main steps used to prepare an API request 9
Figure 5.3: Flow of a typical student action through the stack 10
Figure 6.1: Simplified three-layer architecture of the API 11
Figure 6.2: Major product surfaces in the Next.js application 11
Figure 6.3: Conceptual layout of the item detail page with match panel 11
Figure 8.1: Demonstration outcome board (logical view) 15
Figure 8.2: Relative contribution of default match weights 15
Figure 10.1: Risk-mitigation cycle followed during development 18

LIST OF TABLES

Table 3.1: Comparison of FindIt against broad categories of campus recovery approaches 6
Table 4.1: Core entities stored by the FindIt API 7
Table 4.2: Expected relationship between match signals and recovery usefulness 8
Table 5.1: Default match signal weights and evaluation meaning 9
Table 7.1: Representative unit and integration test cases 13
Table 7.2: Acceptance outcomes for the seeded demonstration path 14
Table 8.1: Walkthrough of the seeded earbuds recovery scenario 15
Table 8.2: Practical reading of default match weights 15
Table 8.3: Practical reading of demonstration results 16
Table 10.1: Project timeline by phase 18
Table 10.2: Summary of tools and libraries used in the project 18
Table 10.3: Project risks and mitigation strategies 19
Table B.1: Extended set of illustrative scenarios used during testing 23
Table B.2: Common issues and suggested fixes 23

TABLE OF CONTENTS

Front Page	i
Acknowledgement	ii
Abstract	iii
Candidate's Declaration	iv
List of Figures	v
List of Tables	vi
Table of Contents	vii
1. Introduction	1
2. Project Objectives and Scope	4
3. Literature Overview	5
4. Data Model Description and Exploration	7
5. Methodologies and Techniques	9
6. Implementation	11
7. Testing and Validation	13
8. Results and Discussion	15
9. Conclusion and Future Work	17
10. Project Management and Development Timeline	18
11. Limitations, Ethical and Practical Considerations	20
12. Appendix A — Source Code	21
13. Glossary of Key Terms and Abbreviations	22
14. Appendix B — Sample Test Cases and User Manual	23
15. References	24

CHAPTER 1
INTRODUCTION

1.1 Background of the Project

University campuses are busy environments in which personal belongings move constantly between classrooms, libraries, hostels, laboratories, and transport hubs. When an item is lost or found, recovery traditionally depends on informal communication: a message in a class group chat, a handwritten notice on a board, or word of mouth. Those channels are familiar, but they do not scale well. Posts disappear under newer messages, people who join a chat late never see an earlier announcement, and searching old threads for a specific item is unreliable.

At the same time, proving ownership often requires a private detail — a scratch on a laptop, a sticker inside a bag, or a partial serial number. Publishing that detail in a public chat weakens the proof and can invite misuse. A campus-oriented information system can keep identifying details private until a claim is reviewed by the finder or an administrator, while still making the public description searchable.

FindIt is a smart campus lost-and-found platform built for this setting. A student reports a lost or found item, searches what has already been approved, and sees a short explanation when another report looks similar. Claims and moderation stay on the record. The project is submitted as a B.Tech CSE third-year minor project in Web Development / Full Stack. Sample people and places in the seed data are fictional.

Modern campuses already run learning-management systems, hostel portals, and library catalogues. A lost-and-found module fits naturally beside those services because it shares the same expectations: sign-in, searchable records, and accountable staff actions. Building FindIt as a standalone full-stack application lets the team demonstrate those expectations without depending on a proprietary campus ERP.

The technical backdrop is a TypeScript monorepo. Separating apps/web from apps/api mirrors how many production teams deploy front ends and APIs independently, while still keeping one repository for a minor-project viva. Documentation in docs/architecture.md and docs/api-reference.md accompanies the code so that examiners can trace a request from the browser to MongoDB.

1.2 Motivation

The first motivation was practical usefulness. Students already know how to post in chats; what they lack is a single place where reports can be filtered by type, category, location, and date, and where similar opposite-type reports are suggested with readable reasons.

The second motivation was to practise a complete full-stack workflow rather than only a user-interface prototype. Many student projects stop at static screens. This project deliberately covers authentication, validation, persistence, role-based access, file upload checks, automated tests, and a seed script for demonstration.

The third motivation was transparency. Match suggestions should not present a mysterious score. FindIt

uses a rule-based heuristic with published weights and written reasons so that a reviewer can understand why two reports were linked — and can still reject a claim when the proof is weak.

A fourth motivation was reproducibility for examination. Seed data, environment examples, and npm scripts make it possible to recreate the same demonstration on another machine. That matters for a viva: examiners should see a stable scenario rather than a fragile one-off database state.

1.3 Problem Statement

Campus lost-and-found communication creates several recurring problems:

- **Scattered announcements:** Group chats and notice boards fragment information so that updates are easy to miss.
- **Awkward privacy:** Owners must often reveal unique marks in public to prove ownership.
- **Weak matching:** Humans may overlook similar reports when wording differs slightly.
- **Missing audit trail:** There is little durable history of who claimed what and who approved it.

1.4 Objectives of the System

- Give students one place to report lost and found items.
- Let anyone search the public (approved) listings.
- Suggest possible matches with an explainable score.
- Let a student claim a found item and let the reporter or an administrator decide.
- Keep administrator actions limited to administrator accounts.

1.5 Scope of the Project

In-Scope: student registration and session-based sign-in; lost and found reports with optional images; search, filters, sorting, and pagination; rule-based match suggestions; claim requests with pending, approved, rejected, and cancelled states; a student dashboard; and an administrator area for moderation, claim review, and activity history.

Out-of-Scope: email or push notifications, campus map integrations, cloud object storage for images, and any claim that match scores are probabilities of recovery. The application is intended for campus demonstration and further academic development.

1.6 Context within the Course: Web Development / Full Stack

This report is submitted as a minor project for Web Development / Full Stack. The course emphasises building end-to-end applications: structured front ends, HTTP APIs, persistent storage, authentication, and honest discussion of limitations. FindIt represents that journey on a concrete campus problem. Ideas such as layered services, cookie-based sessions, and schema validation carry over directly to larger production systems.

1.7 Organisation of the Report

Chapter 2 restates objectives and scope. Chapter 3 surveys related approaches. Chapter 4 describes the data model. Chapters 5 and 6 present methodology and implementation. Chapter 7 documents testing. Chapter 8 discusses results and a sample walkthrough. Chapter 9 concludes and outlines future work. Chapters 10 and 11 cover project management and limitations. Appendices provide source excerpts, a glossary, a user manual, and references.

CHAPTER 2

PROJECT OBJECTIVES AND SCOPE

2.1 Primary Objectives

- Design a clean data model for users, sessions, items, claims, and activity records.
- Implement a layered Express API with validation, authentication, and role checks.
- Deliver a Next.js interface that supports reporting, browsing, matching, claims, and admin moderation without requiring users to call the API manually.

2.2 Secondary Objectives

Secondary goals include transparency of match explanations, privacy of claim proof text, reproducible seed data for class demonstrations, and automated tests that do not depend on the developer's Atlas cluster.

2.3 Scope of the Project

2.3.1 In-Scope (Doable Tasks)

- Student registration, sign-in, and sign-out with an HTTP-only session cookie.
- Lost and found reports, including optional JPEG/PNG/WebP images.
- Search, filters, sorting by newest, and pagination backed by MongoDB.
- Rule-based match suggestions with written reasons.
- Claim workflow and administrator moderation with activity logging.

2.3.2 Out-of-Scope (Non-Doable Tasks)

- Automated email when a claim is reviewed.
- Geospatial maps or building floor plans.
- Managed object storage for images.
- Mobile native applications.

2.4 Expected Deliverables

Expected deliverables are a working monorepo with apps/web and apps/api, environment examples, a seed script, automated tests, architecture and API documentation, this written report, and a demonstration presentation.

2.5 Success Criteria

The project is considered successful if (1) a seeded campus scenario can be demonstrated end-to-end from report to resolved claim, (2) public item JSON never exposes identifying details or claim proof, (3) match reasons are visible on item detail, and (4) automated tests pass against an in-memory database.

CHAPTER 3
LITERATURE OVERVIEW

3.1 Campus Lost-and-Found as an Information Problem

Lost-and-found offices and informal peer networks have long existed on campuses. Digital tools change the problem from physical storage alone to searchability, notification, and trust. Research and practitioner reports on community platforms repeatedly note that visibility without moderation creates spam, while moderation without search creates backlogs. FindIt balances both: reports start as pending, become publicly searchable when approved, and can be resolved through claims.

3.2 Web Application Approaches

Classical campus portals often used server-rendered pages with session cookies. Modern stacks separate a JSON API from a rich client. FindIt follows that pattern: Next.js owns routing and presentation; Express owns rules and persistence. Component libraries and schema validators (Zod) reduce form errors. This mirrors common industry practice for internal tools where correctness and auditability matter more than viral growth features.

3.3 Similarity and Matching Heuristics

Item matching can be approached with machine learning embeddings, fuzzy string matching, or weighted rules. For a minor project with limited labelled data, a transparent heuristic is preferable: category equality, Jaccard overlap of tokenised text and locations, and a decay based on date distance. The score is capped and explained in plain language so that users do not confuse it with a probability.

3.4 Authentication and Session Security

Browser applications commonly store tokens in localStorage or cookies. HTTP-only cookies reduce exposure to script injection on the page. Hashing session identifiers at rest limits damage if the sessions collection is leaked. Role checks must remain on the server: hiding an administrator button is not a security control.

3.5 Current Gaps and Challenges

- Informal chats remain the default despite poor search.
- Many prototypes omit claim privacy and audit logs.
- Aggressive automated matching without explanation reduces trust.
- Image hosting and notifications are often left unfinished in student systems.

3.6 Comparative Study of Existing Approaches

Approach	Search	Privacy of proof	Audit	Fit for demo
Group chats / notice boards	Weak	Poor (public)	None	Familiar but fragile
Simple public CRUD form	Good	Often weak	Limited	Easy, low trust
ML-only matching portal	Strong recall possible	Varies	Varies	Needs data & ops
FindIt (this project)	Filters + heuristic	Proof to reviewers only	Activity log	Designed for viva demo

Table 3.1: Comparison of FindIt against broad categories of campus recovery approaches

3.7 Summary of Literature Review

The literature and practitioner patterns suggest that a successful campus tool must combine moderated visibility, private ownership proof, searchable listings, and explainable suggestions. FindIt adopts those principles with a full-stack implementation suitable for academic demonstration.

In short, FindIt is positioned between disposable chat posts and heavy institutional ERPs. It borrows web-engineering practice — layered services, schema validation, cookie sessions — and applies it to a recovery workflow students recognise immediately.

CHAPTER 4

DATA MODEL DESCRIPTION AND EXPLORATION

4.1 Data Source and Persistence

FindIt does not train on a downloadable market dataset. Its “data” are operational MongoDB documents created by students, administrators, and the seed script. The database name is typically findit in the Atlas or local URI. Restarting the API does not delete saved users, reports, or claims.

4.2 Description of Core Collections

Entity	Key fields	Purpose
User	name, email, studentId, role, passwordHash	Students and administrators
Session	tokenHash, userId, expiresAt	HTTP-only cookie sessions
Item	title, description, type, category, location, eventDate, status, image	Lost/found reports
Claim	itemId, claimantId, proof, status, reviewNote	Ownership requests
ActivityLog	actor, action, resource, time	Moderation and claim history

Table 4.1: Core entities stored by the FindIt API

4.3 Statuses and Visibility

Item statuses include pending, approved, rejected, resolved, and archived. Public browse lists show approved reports. Pending and rejected reports remain visible to the reporter and administrators. Claim statuses include pending, approved, rejected, and cancelled. Public item JSON omits identifying details, private notes, and the reporter id.

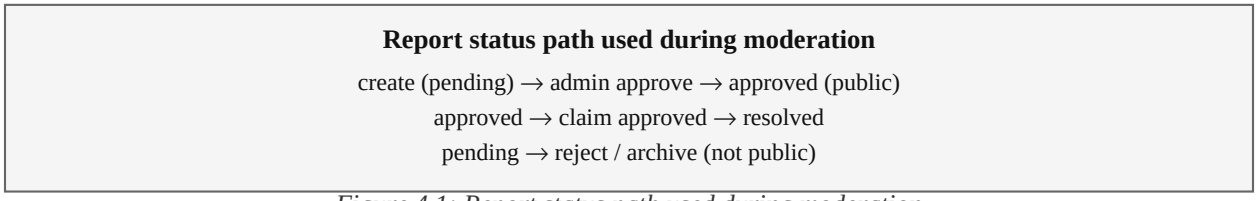


Figure 4.1: Report status path used during moderation

4.4 Seed and Demo Records

The seed script upserts fictional students (for example asha.menon@campus.example) and sample reports such as lost and found earbuds. Demo students share a configured DEMO_STUDENT_PASSWORD. The administrator account comes from ADMIN_NAME, ADMIN_EMAIL, and ADMIN_PASSWORD. Seed:clear removes demo-marked documents without dropping the database or deleting the administrator.

Signal	Represents	Why it matters
Category	Item class (electronics, etc.)	Opposite-type pairs in the same class are likelier
Text overlap	Shared meaningful words	Titles/descriptions often name the object
Location overlap	Shared place words	Loss and find often share a building
Date proximity	Days between events	Same-week events are more plausible

Table 4.2: Expected relationship between match signals and recovery usefulness

4.5 Data Quality Observations

Validation with Zod rejects malformed emails, short passwords, and invalid enums. Image uploads are checked for JPEG, PNG, or WebP signatures and size limits before being written to UPLOAD_DIR. Duplicate demo records are avoided by upserting seed documents rather than blind inserts.

CHAPTER 5

METHODOLOGIES AND TECHNIQUES

5.1 Proposed System Framework

End-to-end framework of FindIt

Browser → Next.js pages → /backend proxy → Express /api
→ middleware → controller → service → repository → MongoDB

Figure 5.1: End-to-end framework of FindIt

The browser never talks to MongoDB and never sees the connection string. Next.js rewrites /backend to the API so cookies belong to the site origin during development.

5.2 Request Preparation and Validation

Main steps used to prepare an API request

1. Parse JSON or multipart body
2. Validate with Zod schemas
3. Resolve session cookie → user + role
4. Enforce route auth and ownership rules

Figure 5.2: Main steps used to prepare an API request

5.3 Layered API Design

Routes and controllers translate HTTP into function calls and status codes. Services hold rules for claims and matching and do not read req or res. Repositories are the only layer that runs Mongoose queries. This separation keeps demonstration code readable and testable.

5.4 Matching Method

For a lost report the matcher considers a limited set of approved found reports (and the reverse). The score adds four weighted signals. The date window defaults to 14 days. Scores below 0.35 are dropped. Results are sorted descending and capped (default eight).

Signal	Default weight	How it is read
Category	0.35	1 if equal, else 0
Text (Jaccard)	0.35	Token overlap of title+description
Location (Jaccard)	0.15	Token overlap of location strings
Date proximity	0.15	Linear decay inside date window
Minimum score	0.35	Candidates below this are omitted

Table 5.1: Default match signal weights and evaluation meaning

5.5 Claims and Moderation

A student may claim an approved found item they did not report. One pending claim is allowed per person per item. The reporter or an administrator may approve or reject. The claimant may cancel a pending claim. Nobody may approve their own claim. Approval updates the claim and marks the item resolved; on Atlas this uses a transaction, with ordered fallback updates on standalone servers.

5.6 Deployment Pipeline for Local Demo

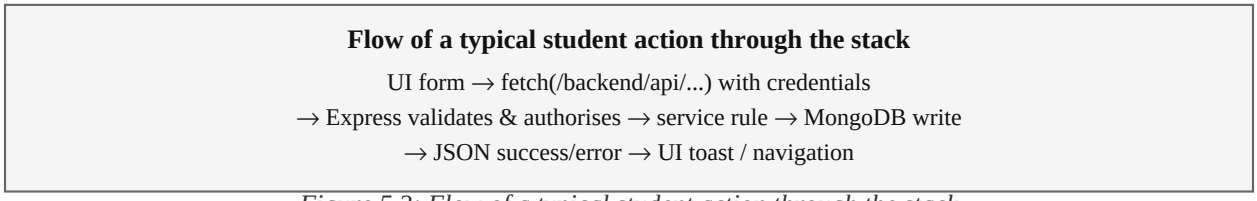


Figure 5.3: Flow of a typical student action through the stack

Developers run npm install, configure MONGODB_URI and admin password, npm run seed, and npm run dev to serve the site on port 3000 and the API on port 4000.

Image handling is part of the methodology even though storage remains local. Uploads are signature-checked for JPEG, PNG, or WebP and constrained by size before the filename is stored on the item. Serving happens through /api/uploads, proxied by Next.js as /backend/uploads, so the browser stays on one origin during development.

Error handling returns structured JSON with safe messages and optional validation details. Stack traces, password hashes, and database credentials are never included in responses. This keeps demonstrations honest about failures without exposing secrets on screen.

CHAPTER 6

IMPLEMENTATION

6.1 Development Environment and Technical Stack

- Next.js App Router, React, TypeScript, Tailwind CSS
- Aceternity UI background lines; ReUI and shadcn-style components
- Express, Mongoose, MongoDB
- Zod, React Hook Form, Vitest, Supertest
- Node.js 20 or newer; npm workspaces-style monorepo scripts

6.2 System Architecture and Pipeline Integration

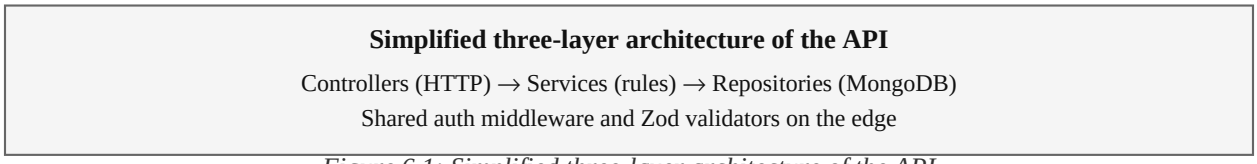


Figure 6.1: Simplified three-layer architecture of the API

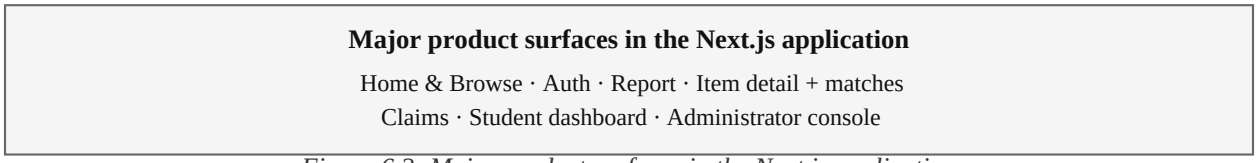


Figure 6.2: Major product surfaces in the Next.js application

6.3 The Application Interface

6.3.1 Browse and Report

Browse lists approved items with search and filters. Creating a report collects title, description, type, category, location, event date, optional identifying details, contact preference, and optional image. New reports start pending until moderated.

6.3.2 Item Detail and Matches

Item detail shows the public description and, when available, ranked match suggestions with scores and reason strings such as shared words or date distance.

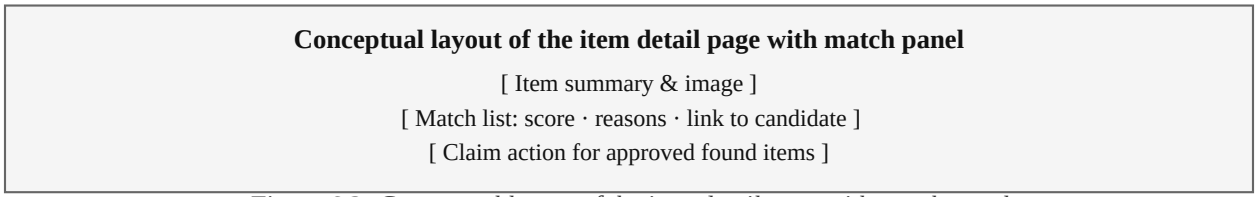


Figure 6.3: Conceptual layout of the item detail page with match panel

6.3.3 Dashboard and Administration

Students see counts derived from their own reports and claims. Administrators open /admin for overview

statistics, moderation actions, claim review, and resource history.

6.3.4 Implementation Notes on the Web App

The web application uses the App Router. Public marketing and browse pages sit beside authenticated dashboard and admin routes. Forms rely on React Hook Form with Zod-aligned validation messages. File upload controls send multipart bodies when an image is attached. Protected pages look for the session cookie, while the API remains the authority for every mutating request.

Concurrent development is started with npm run dev, which launches both processes through concurrently. Typechecking, linting, and production builds are available as separate scripts so that the team can catch regressions before a demonstration.

6.4 Data Handling and Workflow Optimisation

List endpoints paginate with a maximum limit of 50. Match candidate queries are bounded by opposite type, approved status, category or date window, and a candidate limit (default 80) so that scoring does not scan the entire collection on every request.

6.5 Interface Design Principles

The interface prefers clear task-oriented screens over dense dashboards. Public pages emphasise search and recovery. Authenticated areas emphasise the student’s own records. Administrator tools expose moderation without mixing them into the student navigation.

6.6 Security and Privacy Considerations

Passwords are hashed. Session tokens are hashed at rest. Claim proof text is returned only to the reporter and administrators. Activity logs store actor, action, resource, and time — not passwords, raw session tokens, or proof text. A student who tampers with a request body cannot escalate to administrator.

Representative public routes include health, registration, login, item search, item stats, and visible item details. Authenticated students create reports and claims. Administrators use /api/admin routes for overview counts, moderation, claim review, and resource history. The full catalogue appears in the project API reference.

CHAPTER 7

TESTING AND VALIDATION

7.1 Testing Approach and Levels

Testing combines automated API tests with a scripted manual demonstration. Automated tests use Vitest and Supertest with an in-memory MongoDB and do not read or write the developer's MONGODB_URI.

7.2 Unit and Integration Testing

ID	Focus	Expected result
TC-01	Register + login cookie	Session established; /me returns student
TC-02	Create item pending	Item stored; not listed on public browse
TC-03	Admin approve item	Item appears in public list
TC-04	Match earbuds scenario	Opposite-type candidate scored with reasons
TC-05	Claim approve resolves	Claim approved; item status resolved
TC-06	Public JSON privacy	No identifyingDetails / proof in public payload

Table 7.1: Representative unit and integration test cases

7.3 Validation Strategy

Schema validation rejects invalid enums and missing required fields with safe error messages. Health returns 503 with DATABASE_UNAVAILABLE when MongoDB is down. Auth routes are rate limited outside tests.

7.4 Interface and Usability Checks

Manual checks confirm that navigation between browse, report, detail, dashboard, and admin remains understandable for a first-time demonstrator, and that empty states are readable when the database has not been seeded.

7.5 Demonstration Checklist Validation

The README demonstration path — seed, sign in as a demo student, open the lost earbuds report, read the match explanation, moderate a pending found item, and complete a claim — is treated as an acceptance checklist for viva preparation.

Step	Outcome
Seed admin + demo students	Accounts available; fictional reports upserted
Home stats	Non-zero counts after seed
Match explanation visible	Reasons list includes heuristic disclaimer
Claim → approve	Item becomes resolved

Table 7.2: Acceptance outcomes for the seeded demonstration path

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Overall System Behaviour

After seeding and starting both processes, the platform behaves as a coherent recovery workspace: public browse shows approved items, pending reports stay private to owners and admins, matches appear on eligible item pages, and claims leave an auditable trail.

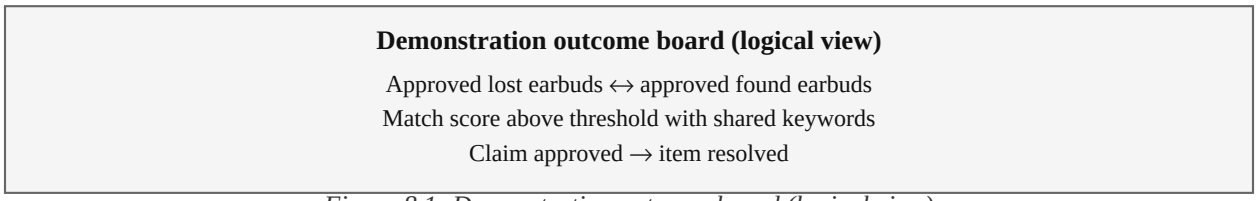


Figure 8.1: Demonstration outcome board (logical view)

8.2 Sample Walkthrough

Step	Actor	Action	Result
1	Operator	npm run seed	Demo data ready
2	Student Asha	Open lost earbuds	Sees match to found report
3	Admin	Approve pending found (if any)	Listing becomes public
4	Claimant	Submit claim with proof	Claim pending
5	Reporter/Admin	Approve claim	Item resolved

Table 8.1: Walkthrough of the seeded earbuds recovery scenario

8.3 Match Explanation Discussion

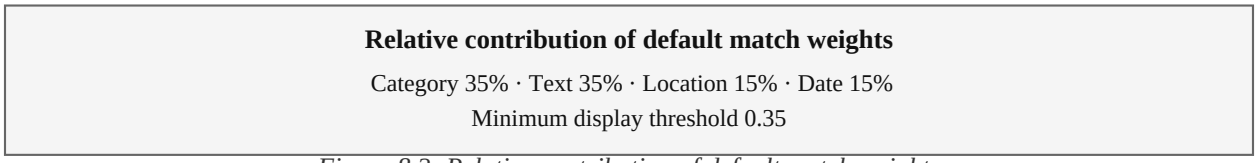


Figure 8.2: Relative contribution of default match weights

Signal	Approx. share	Reading
Category	35%	Strong gate when classes differ
Text	35%	Names the object in overlapping words
Location	15%	Supports same-building stories
Date	15%	Prefers events close in time

Table 8.2: Practical reading of default match weights

8.4 Discussion of Results

Observation	Interpretation
Explainable scores	Reviewers can contest weak matches
Pending before public	Reduces spam on browse
Proof not public	Protects ownership secrets
Heuristic limits	May miss paraphrases outside window

Table 8.3: Practical reading of demonstration results

8.5 Behaviour Outside Ideal Conditions

If MongoDB is unreachable, the API exits or health fails rather than inventing fake data. If images are uploaded while the API is down, the web proxy cannot serve /backend/uploads. Match suggestions can miss similar items outside the category and date window by design.

8.6 Interpretation for Examiners

The results should be read as evidence of a working educational system, not as a claim of production readiness for an entire university. Success is the coherent loop — report, moderate, match, claim, resolve — with privacy boundaries that can be shown live. Residual limitations are listed openly in Chapter 11 so that future teams know where to extend.

Compared with a notebook-only prototype, the presence of seeded accounts, HTTP cookies, and administrator history makes the demonstration closer to how internal campus tools actually behave. That realism is an intentional result of the full-stack methodology.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

FindIt demonstrates that a campus lost-and-found workflow can be implemented as a clear full-stack system: moderated listings, private claim proof, explainable matching, and administrator oversight. The project meets its educational goals by exposing students to authentication, layered APIs, validation, testing, and honest product boundaries.

9.2 Future Work

- Email or in-app notifications when a claim is reviewed.
- A map or building list for locations.
- Object storage for images instead of local disk.
- A richer audit view for administrators.
- Optional stronger text similarity while keeping explanations.

9.3 Extension towards Richer Campus Services

The same architecture could later host related campus services — for example locker returns or lab-equipment loans — by reusing sessions, roles, and activity logging while replacing the item schema and match rules.

CHAPTER 10

PROJECT MANAGEMENT AND DEVELOPMENT TIMELINE

10.1 Development Methodology

Work followed an iterative full-stack approach: agree on the recovery workflow, model collections, implement API routes with tests, build corresponding Next.js screens, seed demo data, and refine documentation for viva demonstration.

10.2 Project Timeline

Phase	Focus	Outcome
1	Requirements & scope	Problem, objectives, out-of-scope list
2	Data model & API	Mongoose schemas, layered services
3	Matching & claims	Heuristic scorer, claim states
4	Web interface	Browse, report, detail, admin
5	Testing & seed	Vitest suite, demo script
6	Report & PPT	Academic documentation

Table 10.1: Project timeline by phase

10.3 Resource and Tool Summary

Tool	Role
Node.js / npm	Runtime and package management
Next.js / React / TS	Web application
Express / Mongoose	HTTP API and ODM
MongoDB Atlas/local	Persistence
Zod / RHF	Validation and forms
Vitest / Supertest	Automated tests
Git	Version control

Table 10.2: Summary of tools and libraries used in the project

10.4 Risk Management



Figure 10.1: Risk-mitigation cycle followed during development

Risk	Mitigation
DB connection failures	Fail fast; health 503; no fake data
Role escalation	Server-side role checks; register forces student
Match overtrust	Disclaimer in reasons; human claim review
Image disk limits	Type/size checks; future object storage noted
Demo flakiness	Idempotent seed upserts; clear README steps

Table 10.3: Project risks and mitigation strategies

CHAPTER 11

LIMITATIONS, ETHICAL AND PRACTICAL CONSIDERATIONS

11.1 Limitations of the Current System

- Images are stored on the API machine; object storage is not wired up.
- Matching is a capped heuristic and can miss paraphrased reports.
- The app does not send email.
- Standalone MongoDB may not support transactions; ordered fallback is used.

11.2 Ethical Considerations

Lost-and-found systems handle sensitive personal context. FindIt therefore keeps identifying details and claim proof out of public JSON, restricts administrator routes, and labels match scores as similarity estimates. Seed data uses fictional identities so that demonstrations do not expose real student information.

11.3 Practical Deployment Considerations

A real campus deployment would additionally require institutional authentication, backup policies, abuse reporting, retention rules for images, and monitored hosting. Those operational concerns are acknowledged as future work rather than claimed as finished.

11.4 Comparison with Informal Human Judgement

Human judgement in chats can be nuanced but is inconsistent and hard to audit. FindIt does not replace human review of claims; it structures the evidence and the decision so that outcomes are repeatable and inspectable.

CHAPTER 12

APPENDIX A — SOURCE CODE

A.1 Repository Layout

```
apps/web          Next.js interface
apps/api          Express API, models, tests, seed
docs              Architecture and API reference
```

A.2 Match Scoring (excerpt)

```
const raw =
  config.weights.category * categoryScore +
  config.weights.text * textScore +
  config.weights.location * locationScore +
  config.weights.date * dateScore;
const score = Math.round(raw * 100) / 100;
if (score < config.minScore) return null;
```

A.3 Default Matching Configuration (excerpt)

```
weights: { category: 0.35, text: 0.35, location: 0.15, date: 0.15 },
dateWindowDays: 14,
minScore: 0.35,
candidateLimit: 80,
resultLimit: 8,
```

A.4 Request Path (conceptual)

```
Browser -> Next.js -> /backend proxy -> Express route
       -> middleware -> controller -> service
       -> repository -> MongoDB
```

A.5 Key npm Scripts

```
npm install
npm run seed
npm run dev          # web :3000, api :4000
npm run test
npm run seed:clear
```

CHAPTER 13

GLOSSARY OF KEY TERMS AND ABBREVIATIONS

FindIt
Smart campus lost-and-found platform developed in this project.

Item
A lost or found report document with status and metadata.

Claim
A request asserting ownership of an approved found item.

Match score
Heuristic similarity estimate with written reasons; not a probability.

Jaccard overlap
Size of token intersection divided by union; used for text/location.

Session cookie
HTTP-only findit_session cookie holding a random session id.

Repository
API layer that alone executes Mongoose queries.

Moderation
Administrator approve/reject/archive/resolve actions on reports.

Seed script
Idempotent loader of fictional demo users and reports.

Vitest / Supertest
Tools used for automated API tests.

Zod
Schema validation library for request bodies and config.

ODM
Object-Document Mapper; Mongoose maps schemas to MongoDB.

CHAPTER 14

APPENDIX B — SAMPLE TEST CASES AND USER MANUAL

B.1 Extended Sample Test Cases

Profile	Purpose
Default seed earbuds	Baseline match + claim path
New student pending found	Confirm browse hides pending
Admin reject report	Confirm not public; owner can still see
Second claim same user	Reject duplicate pending claim
Self-claim attempt	Blocked by service rules
Invalid image type	Upload rejected safely

Table B.1: Extended set of illustrative scenarios used during testing

B.2 User Manual — Running the Application

- Install Node.js 20+ and npm; clone the repository.
- Run npm install from the project root.
- Copy apps/api/.env.example to .env and set MONGODB_URI and ADMIN_PASSWORD.
- Copy apps/web/.env.example to .env.local (proxy defaults are already suitable).
- Run npm run seed, then npm run dev.
- Open http://localhost:3000; API health at http://localhost:4000/api/health.
- Sign in as asha.menon@campus.example with DEMO_STUDENT_PASSWORD.
- Walk the earbuds match and claim path; use admin credentials for moderation.

B.3 Troubleshooting FAQ

Issue	Suggested fix
API exits immediately	Check MONGODB_URI and Atlas IP allow list
Site loads but actions fail	Ensure API is running; check API_PROXY_TARGET
Seed rejects admin password	Replace change-me-admin-password
Images missing	API must serve /uploads via /backend proxy

Table B.2: Common issues and suggested fixes

CHAPTER 15
REFERENCES

1. Vercel Inc., “Next.js Documentation,” [Online]. Available: https://nextjs.org/docs

2. OpenJS Foundation, “Express — Node.js web application framework,” [Online]. Available: https://expressjs.com/

3. MongoDB Inc., “MongoDB Manual,” [Online]. Available: https://www.mongodb.com/docs/manual/

4. Automattic, “Mongoose ODM,” [Online]. Available: https://mongoosejs.com/docs/

5. Zod Documentation, [Online]. Available: https://zod.dev/

6. React Hook Form, [Online]. Available: https://react-hook-form.com/

7. Vitest, [Online]. Available: https://vitest.dev/

8. Tailwind Labs, “Tailwind CSS,” [Online]. Available: https://tailwindcss.com/docs

9. Fielding, R. T., Architectural Styles and the Design of Network-based Software Architectures, Doctoral dissertation, UC Irvine, 2000.

10. OWASP Foundation, “Session Management Cheat Sheet,” [Online]. Available: https://cheatsheetseries.owasp.org/

11. MDN Web Docs, “HTTP cookies,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies

12. Punjabi University, Patiala — Department of Computer Science & Engineering, Minor Project Guidelines (course reference).